CHAPTER 6

Tree-Based Methods

In the world of machine learning, tree-based methods are very useful. They are relatively simple to explain and easy to visualize. In some cases with machine learning models (notably complex neural networks), the trained model can effectively be a black box whose inner workings are too complex for us to explain simply. Tree-based models, on the other hand, can be a lot more intuitive for the average user.

In this chapter, we look at how tree-based models work at a high level by focusing first on decision trees. We then dive into the basic mechanics of how they work and some positive and negative attributes associated with them. We also touch on different types of tree-based models like conditional inference trees and random forests. To give you a preview, decision trees are as simple as "if-then" statements related to data. Conditional inference trees work in a similar manner but with slightly different statistical underpinnings. Random forests can be complicated mathematically, but generally boil down to a collection of different tree models being asked to vote on a result. All of these types can be used for regression modeling (regression trees) or classification modeling (classification trees). Many can be used for both purposes and are called classification and regression trees (CART) models.

A Simple Tree Model

Let's begin by looking at an example of a set of data that describes my bike races this year. We could have a variety of parameters just related to the weather. If I had a robust enough sample of data and I had a bike race coming up where I knew the forecast to a reasonable degree, should I expect a good or bad result to my bike race? Table 6-1 lists a few different weather factors and what my corresponding race result was.

Table 6-1. Example weather conditions and good race results

Week	Sky condition	Wind speed	Humidity	Good result
1	cloudy	low	high	yes
2	rainy	low	normal	yes
3	sunny	high	normal	yes
4	cloudy	high	high	yes
5	cloudy	low	normal	yes
6	rainy	high	high	no
7	rainy	high	normal	no
8	cloudy	high	normal	yes
9	sunny	low	high	no
10	sunny	low	normal	yes
11	rainy	low	normal	yes
12	sunny	low	high	no
13	sunny	high	high	no

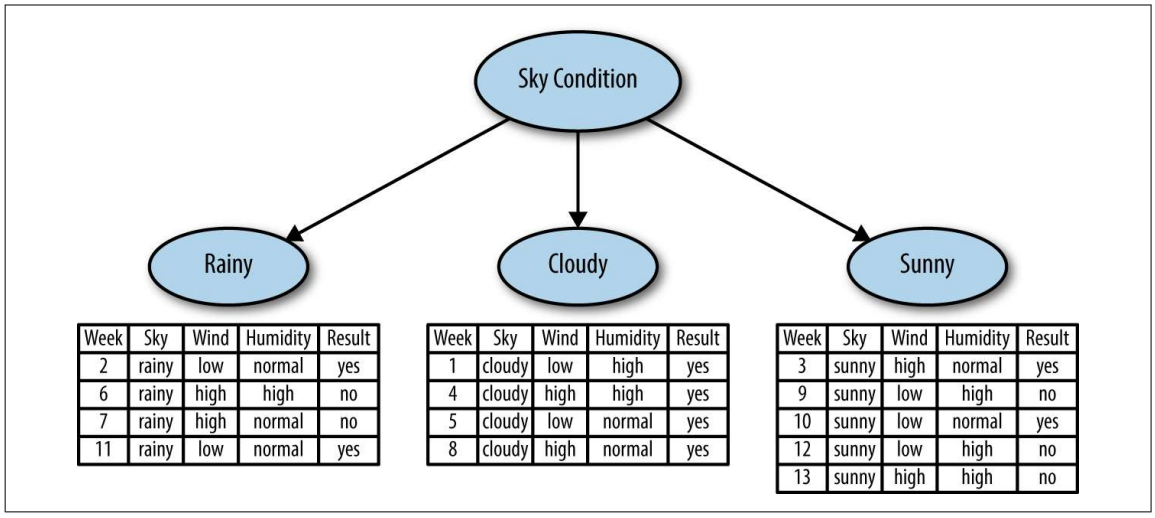
Suppose that the race for week 14 is going to have the following weather:

- Sky Condition: rainy
- Wind speed: low
- Humidity: high

What we might do, if this data were in a spreadsheet, would be to filter our data on those exact conditions and see what the results look like. A tree-based model does basically the same thing. It subsets the data by certain criteria and then builds a tree so that when we have new data, it follows the branches of the tree to a result. Figure 6-1 takes the data from Table 6-1 and represents it as a tree, with the first split being on the `Sky Condition` variable.

Figure 6-1 shows three leaves in the tree: `Sky Condition = rainy`, `Sky Condition = cloudy`, and `Sky Condition = sunny`. For each of these subsets, you can see how the data looks. The response you want to model is whether I'm going to have a good result in my race. A decision tree looks at these subsets and examines whether the `Result` variable contains all of one particular class. For cloudy races, I have a good result being `yes` for each one. This indicates that the subset is pure and that you don't need to split it any further. However, the sunny and rainy subsets have a mix of `yes` and `no` results. You'll want to split these further to get a subset with higher purity.

Purity is defined as how many positive or negative examples you have that you're trying to model for (in this case, `Result`) out of the total values in the table. You want to continue to split the tree until you have as many pure leaves as possible.



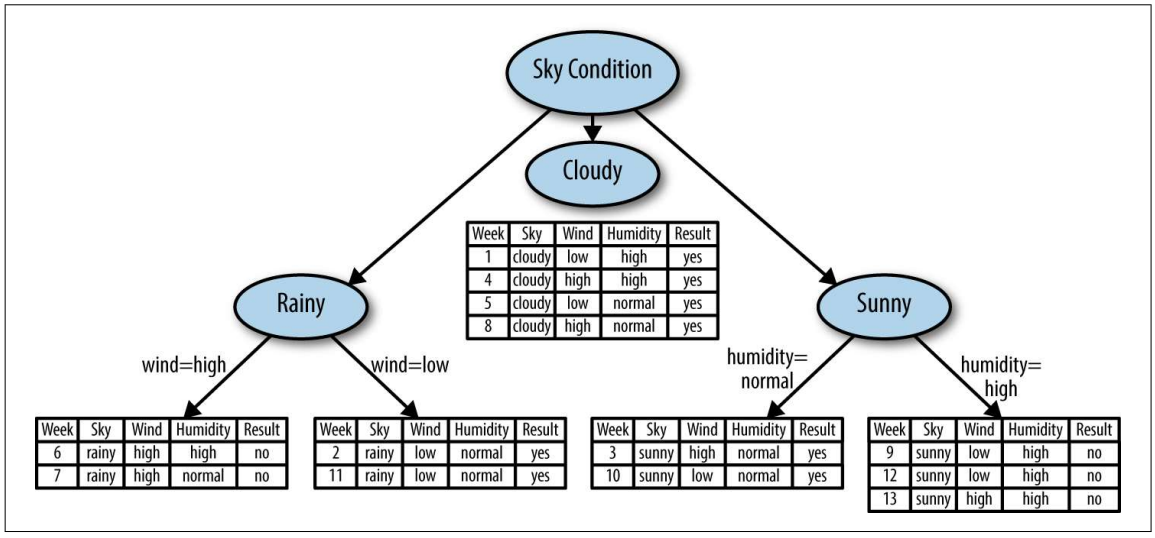
Week	Sky	Wind	Humidity	Result
2	rainy	low	normal	yes
6	rainy	high	high	no
7	rainy	high	normal	no
11	rainy	low	normal	yes

Week	Sky	Wind	Humidity	Result
1	cloudy	low	high	yes
4	cloudy	high	high	yes
5	cloudy	low	normal	yes
8	cloudy	high	normal	yes

Week	Sky	Wind	Humidity	Result
3	sunny	high	normal	yes
9	sunny	low	high	no
10	sunny	low	normal	yes
12	sunny	low	high	no
13	sunny	high	high	no

Figure 6-1. An example of how a decision tree subsets data

As Figure 6-2 demonstrates, by splitting up the rainy and sunny subsets further (in this case by wind and humidity, respectively), there are now five terminal points to the tree. You can read this tree like an "if-then" statement, starting at the top. First, let's revisit what you want to predict. You want to know if I'm going to have a good race result if it's rainy, with low wind speed, and high humidity. Begin at the top of the tree and move along the path of `Sky Condition = rain`, and then split on the wind speed being low. That brings you into a bucket that has all of its data having a good result status of `yes`. So, based on the conditions that you wanted to predict initially, through the use of a decision tree, you can predict that I'll have a good race result!



Week	Sky	Wind	Humidity	Result
1	cloudy	low	high	yes
4	cloudy	high	high	yes
5	cloudy	low	normal	yes
8	cloudy	high	normal	yes

Week	Sky	Wind	Humidity	Result
6	rainy	high	high	no
7	rainy	high	normal	no

Week	Sky	Wind	Humidity	Result
2	rainy	low	normal	yes
11	rainy	low	normal	yes

Week	Sky	Wind	Humidity	Result
3	sunny	high	normal	yes
10	sunny	low	normal	yes

Week	Sky	Wind	Humidity	Result
9	sunny	low	high	no
12	sunny	low	high	no
13	sunny	high	high	no

Figure 6-2. This decision tree splits further based on wind speed or humidity to create more resultant tables (or leaves) at the ends of the tree that have a higher purity than before

Deciding How to Split Trees

In the previous example, you started with a dataset that you wanted to model. There were many features and a response against which you wanted to model (result as a function of sky condition, wind speed, and humidity). You began by looking at how a decision tree works by splitting on the variable `Sky Condition`, but why did we choose that one? Why not choose a different variable to split on instead? A decision tree wants to maximize the "purity" of its splits. Let's take a look at Figure 6-3, which revisits the first cut of the tree from earlier.

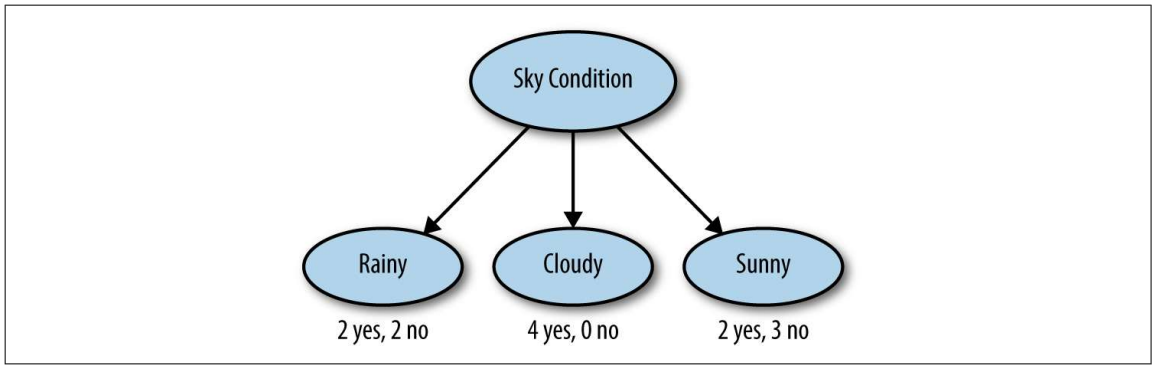


Figure 6-3. When splitting decision trees, you want as pure of leaf nodes as possible (in this case, Cloudy is the most pure leaf, followed by Sunny, then Rainy; we need to split our tree further on the impure leaf nodes to get a better tree model)

In Figure 6-3, the `cloudy` leaf starts out as 100%. That means 100% of the data in that cut is all `yes` data and requires no further subsetting to get to a pure set. The `sunny` and `rainy` leaves do require splitting, however. Let's contrast with starting out with a tree based on the `Wind Speed` variable, as shown in Figure 6-4.

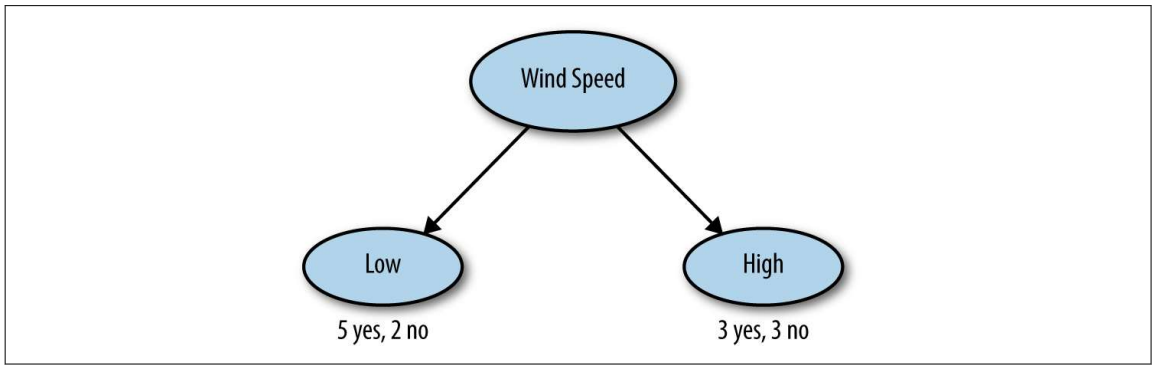


Figure 6-4. Splitting the data based on Wind Speed does not yield results with a high purity factor

In Figure 6-4, none of the initial leaves are totally pure. One leaf is 75% pure, and another is 50%. These both require additional splits of the data. You want to achieve as much purity as you can, and having a tree with one leaf that's 100% pure at the

outset points to that variable as the place to start versus the others. Virtually all decision tree algorithms will have this functionality built in, so you don't really ever need to worry about determining an initial split of the data, but it's good practice to know how the underlying mechanics of the tree algorithms work.

Tree Entropy and Information Gain

A more mathematical way to represent the purity of each subset in the tree is to measure its entropy. This is a way to measure how likely you are to get a positive element if you randomly select from a particular subset. The equation for entropy is as follows:

$$H(S) = -p_{+}\log_2 p_{+} - p_{-}\log_2 p_{-}$$

In plain English, this equation states that the entropy for a given set of data is a function of how many positive cases $p+$ we have in total, times the logarithm (note the base two) of the same value, and then subtracting the number of negative cases in the same way. Recall that positive in this case would be a "yes" result, and negative would be a "no" result in the original example data in Table 6-1. A good way to see this in action is to consider how you can apply it to the tree that you split on `Sky Condition`:

$$H(\text{rainy}) = -\frac{2}{4}log_2\frac{2}{4} - \frac{2}{4}log_2\frac{2}{4} = 1$$

$$H(\text{cloudy}) = -\frac{4}{4}log_2\frac{4}{4} - \frac{0}{4}log_2\frac{0}{4} = 0$$

$$H(\text{sunny}) = -\frac{2}{5}log_2\frac{2}{5} - \frac{3}{5}log_2\frac{3}{5} = 0.97$$

The values you get for rainy and sunny are both 1. This means that the sample is about as impure as you could get and that it would require further splitting, whereas the sample you have for cloudy is totally pure, and the entropy for that is 0. We are getting away with a mathematical trick for this case, because technically speaking the logarithm of zero should be negative infinity, but we are "canceling" that out by multiplying by zero anyway.

Although the entropy of the individual leaves is nice, the algorithm determines the most useful features to split on first by finding the features with the highest *gain*. Gain is a measurement of how relevant the feature is on a scale from 0 (least useful) to 1 (most useful) and is defined by

$$\text{Gain} = H(S) - \Sigma\frac{S_V}{S}H(S_V)$$

where V is the possible values of the feature, S is the number of total points in the leaf, and SV is the subset for which we have our possible values of the feature. Let's run this specifically on the tree that's split by the `Wind Speed` feature:

$$H(\text{wind}) = -\frac{9}{14}log_2\frac{9}{14} - \frac{5}{14}log_2\frac{5}{14} = 0.94$$

$$H(\text{low}) = -\frac{6}{8}log_2\frac{6}{8} - \frac{2}{8}log_2\frac{2}{8} = 0.81$$

$$H(\text{high}) = -\frac{3}{6}log_2\frac{3}{6} - \frac{3}{6}log_2\frac{3}{6} = 1$$

$$\text{Gain(Wind)} = 0.94 - \frac{8}{14} * 0.81 - \frac{6}{14} * 1 = 0.049$$

An easy way to do this in R is to use the `varImpPlot()` function from the `caret` package. Although this specific function uses a slightly different mathematical computation than Gain specifically, the result is the same. With this plot, you can see that `Sky Condition` is the most important factor, followed by `Humidity` and then `Wind Speed`. If you had a dataset with many more variables and wanted to see which ones a particular tree algorithm thought were the most important, you could use the `VarImp Plot()` function to get a quick glance to see how to split the tree from the top down:

```
library(caret)
library(randomForest)

cycling <- read.table("cycling.txt", sep = "\t", header = T)

fit <- randomForest(factor(Result) ~ ., data = cycling[2:5])

varImpPlot(fit)
```

Pros and Cons of Decision Trees

So far we've seen one good attribute of decision trees: they are easy to explain. By starting with some sample data, we were able to represent it in a tree-like fashion and then simply walk through the various splits in the data to come to a conclusion. When you train a tree-based model, the underlying mechanics will use the same functionality, so it's very easy to explain how the model itself works.

We've also seen that trees can handle irrelevant data automatically (i.e., when the gain metric is zero). This eliminates the need to be careful in determining which features you want to model against because the tree will almost always select the best attributes for you. Feature selection is a large part of most modeling procedures, and having tree-based models do it for you is a big headache saver.

Another advantage of trees is that they are pretty fast at computation after they've been tuned. In fact, after the tree has been well tuned, the resultant model tends to be quite succinct. This aids not only in explanation, but keeping the model itself relatively simple. It's easy to tell when a tree is overfit when it looks like it's becoming too specific or if there are lots and lots of small branches in the model. Finally, tree-based models can also handle missing or outlier data, again saving you the headache of having to do tedious quarantine procedures that might be more common with other models.

Tree Overfitting

The downsides to trees are that they can be very sensitive to initial conditions or variations in the data. Because we are splitting on attributes and likely value ranges in our data, if we alter values slightly, we could be eliminating entire branches of our model. Another issue with trees is that they follow axis-aligned splits of the data. If we had a tree for some example data, where the output was as follows, we would have a corresponding plot of the data that looks like that shown in Figure 6-5:

- If *X* is less than 2.5, the result is 25.
- If *X* is greater than 2.5, and *Y* is less than 15, the result is 9.
- If *X* is greater than 2.5, and *Y* is greater than 15, the result is 3.14.

You can see immediately that trees split the data into boxes, given how they're designed in relation to features. This works better for some versions of data than others. Data that is easily split into boxes relative to the axes that you are splitting by will work better, like in the case of the `iris` dataset illustrated in Figure 6-6.